

Lab 05 · Las fichas que se apuntan entre sí

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

90 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

Que una relación entre dos tablas se declara con dos anotaciones, y que la decisión de cuándo traer lo que hay al otro lado cambia el número de consultas: **1 SELECT con LAZY**, **4 con EAGER**, contados en pantalla.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. Las fichas que se apuntan, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · La casilla que apunta	3
3.2. Paso 2 · El lado espejo	5
3.3. Paso 3 · Contar los viajes: LAZY contra EAGER	6
3.4. Paso 4 · El error que todo el mundo se encuentra	7
3.5. Paso 5 · Un nombre de método que cruza la relación	8
4. Lo que aprendiste	10
5. Para profundizar	10
6. Antes de cerrar	10

1 Antes de empezar

1.1 Qué vas a lograr

En el Lab 04 la ficha estaba sola en su cajón. Hoy dos fichas se apuntan entre sí: un trámite pertenece a un contribuyente, y un contribuyente tiene muchos trámites.

Vas a declarar esa relación con dos anotaciones, vas a **contar las consultas** que se disparan según cómo la declares, y vas a provocar a propósito el error que todo el mundo se encuentra la primera vez: el `LazyInitializationException`. Al final vas a escribir un método cuyo **nombre cruza la relación** y verás salir un `join` que nadie escribió.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
El Lab 04 hecho	Sabes qué es <code>@Entity</code> y qué hace un repositorio	Es imprescindible aquí
Estar en la carpeta del lab	<code>cd labs/lab-05-relaciones/practica</code>	El <code>cd</code> no da error
Nada más	La base viaja dentro del repositorio	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa: el compilador ignora la sangría. Si una línea se parte, el editor te la marca. El código completo está en `labs/lab-05-relaciones/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-05-relaciones/practica
./mvnw spring-boot:run
```

Escucha en el **8087** y su PostgreSQL en el **55434**. **Párala siempre con Ctrl+C**: los dos errores de arranque del Lab 04 —el puerto ocupado y el candado tomado— salen aquí igual, con los números de este lab.

2 El caso

El archivador de la DGT tiene ahora **dos cajones**: el de contribuyentes y el de trámites. Y cada trámite es **de alguien**.

2.1 Las fichas que se apuntan, que es la metáfora de este laboratorio

Una ficha lleva escrito el número de otra.

En la ficha de cada trámite hay una casilla que dice **a qué contribuyente pertenece**, con su número. Esa casilla es la columna `contribuyente_id`, y la ficha que la lleva escrita es **la dueña de la relación**.

La ficha del contribuyente **no lleva escritos** los números de sus trámites — sería una lista que crece sin fin. Lo que tiene es una forma de decir «*mis trámites son los que me apuntan a mí*». Eso es `mappedBy`, y por eso `mappedBy` va siempre **en el lado que NO tiene la casilla**. Y ahora la decisión del día, que es de logística del archivista:

Cuando te traigo la ficha de un trámite, ¿te subo también la del contribuyente?

- **LAZY** — «te traigo el trámite; si además quieres al contribuyente, me lo pides y bajo otra vez». Un viaje ahora, y otro sólo si hace falta.
- **EAGER** — «te traigo el trámite y, por si acaso, subo también al contribuyente». Siempre.

Con seis trámites, la diferencia entre las dos es **un viaje o cuatro**. Lo vas a contar tú en el paso 4.

3 Los pasos

3.1 Paso 1 · La casilla que apunta

Qué vamos a hacer

Declarar, en la ficha del trámite, a qué contribuyente pertenece.

Para entenderlo mejor

Añadir a la ficha del trámite la casilla *«contribuyente nº ____»*. Es una casilla como las demás, sólo que lo que lleva dentro es **el número de otra ficha**.

El problema

Sin relación declarada tendrías que guardar el número a mano —un `Long contribuyenteId`— y, cada vez que quisieras el contribuyente, ir a buscarlo tú. Funciona, y significa escribir esa búsqueda en todos los sitios que la necesiten.

La alternativa, y por qué no

- **Guardar el id suelto** (`Long contribuyenteId`): simple, y te deja sin navegación. Es lo que se hace **a propósito** cuando las dos tablas viven en servicios distintos — lo verás en el Lab 14, y ahí es lo correcto.
- **Declarar la relación**, que es lo de aquí: escribes el objeto, no el número, y puedes navegar de un lado al otro.

Y dentro de declararla, la decisión que importa:

- `fetch = EAGER` es **el valor por defecto de la especificación JPA** para `@ManyToOne`. Trae siempre el otro lado.
- `fetch = LAZY`, que es lo que se pone aquí **contra el valor por defecto**, y a propósito: `EAGER` en un `@ManyToOne` es la fábrica del problema del Lab 06. Lo vas a medir en el paso 4.

Se pega

En `practica/src/main/java/cl/dgt/relaciones/entities/Tramite.java`, **entre los campos**:

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "contribuyente_id", nullable = false)
private Contribuyente contribuyente;
```

Dos anotaciones y cada una dice una cosa:

- `@ManyToOne` — muchos trámites apuntan a un contribuyente.
- `@JoinColumn(name = "contribuyente_id")` — **cuál es la casilla**. Si lo omites, Hibernate se inventa el nombre por convención; escribirlo deja el vínculo declarado y no dependiendo de una regla que hay que recordar.

Lo que vas a ver

```
=== 2 · NAVEGAR · tramite -> contribuyente ===
Hibernate:
  select
    t1_0.id, t1_0.contribuyente_id, t1_0.estado, t1_0.fecha, t1_0.tipo
  from
    tramite t1_0
  where
    t1_0.id=?
  trámite cargado: Tramite{id=1, tipo='Declaración F29', estado='RECIBIDO',
↪ fecha=2026-03-10}
  --- todavía NO se ha tocado el contribuyente ---
Hibernate:
  select
    c1_0.id, c1_0.razon_social, ...
```

Ahí está LAZY en dos líneas. Primero llegó el trámite y **sólo el trámite**. El segundo `select` no salió hasta que el código pidió el contribuyente.

Vas bien si...

Ves **dos** bloques Hibernate: separados por la línea --- todavía NO se ha tocado el contribuyente ---. Si vieras uno solo con un join, tendrías EAGER puesto.

Si te atascas

1 · Schema-validation: missing column [contribuyente_id]

El nombre del @JoinColumn no coincide con la columna de la tabla. La tabla ya existe: la clase se adapta a ella, no al revés.

2 · cannot find symbol: class JoinColumn

Falta el import de jakarta.persistence.JoinColumn.

3.2 Paso 2 · El lado espejo

Qué vamos a hacer

Que desde el contribuyente se puedan ver sus trámites.

Para entenderlo mejor

La ficha del contribuyente no lleva escrita la lista de sus trámites. Lo que dice es: «*mis trámites son los que llevan mi número en su casilla*». El archivista sabe buscarlos.

El problema

Si el contribuyente guardara la lista, habría **dos sitios** donde vive la misma verdad: la casilla del trámite y la lista del contribuyente. En cuanto los dos se puedan editar, se pueden contradecir.

La alternativa, y por qué no

- **No tener lado inverso** y consultar siempre por el repositorio: es lo correcto cuando la colección es grande — una oficina con cien mil trámites no quiere una List que se pueda cargar entera por accidente.
- **Tener lado inverso con mappedBy**, que es lo de aquí: cómodo para colecciones pequeñas, y **sin duplicar la verdad**, porque mappedBy dice exactamente que la columna vive en la otra tabla.

Se pega

En practica/src/main/java/cl/dgt/relaciones/entities/Contribuyente.java, **entre los campos**:

```
@OneToMany(mappedBy = "contribuyente")
private List<Tramite> tramites = new ArrayList<>();
```

mappedBy = "contribuyente" nombra el CAMPO de la otra clase, no la columna de la tabla. Es el error más repetido de esta anotación: se escribe contribuyente_id y no funciona.

Lo que vas a ver

tiene 2 trámites:

```
Tramite{id=1, tipo='Declaración F29', estado='RECIBIDO', fecha=2026-03-10}
Tramite{id=2, tipo='Certificado de situación', estado='EMITIDO', fecha=2026-07-02}
```

Vas bien si...

Desde un contribuyente puedes listar sus trámites, y salen los dos.

Si te atascas

1 · La aplicación no arranca y habla de una tabla contribuyente_tramites.

Es el error clásico de este paso. Te falta mappedBy. Sin él, JPA cree que la relación es unidireccional y espera **una tabla intermedia** que no existe. Como el lab usa ddl-auto: validate, lo dice al arrancar.

2 · mappedBy reference an unknown target entity property

Pusiste en mappedBy el nombre de la columna en vez del nombre del campo. Va "contribuyente", que es como se llama el campo en Tramite.

3.3 Paso 3 · Contar los viajes: LAZY contra EAGER

Qué vamos a hacer

Traer seis trámites y **contar las consultas**. Primero con LAZY, después cambiando a EAGER.

Para entenderlo mejor

Seis trámites sobre la mesa. Con LAZY, el archivista hizo **un viaje**. Con EAGER, subió también las fichas de los contribuyentes de cada uno: **cuatro viajes**.

El problema

La diferencia no se ve leyendo el código: las dos versiones se escriben igual y funcionan igual. **Solo se ve contando**. Y con seis filas la diferencia es irrelevante; con seis mil, decide si la página carga o no.

Se corre

Arranca y busca en la consola las dos marcas:

```
>>>>> EMPIEZA EL CONTEO – cuenta los 'Hibernate:' hasta la marca de fin
...
<<<<< FIN DEL CONTEO – 6 trámites traídos
```

Lo que vas a ver

Con fetch = LAZY, que es lo que tienes puesto:

```
>>>>> EMPIEZA EL CONTEO
Hibernate:
```

```

select
    t1_0.id, t1_0.contribuyente_id, t1_0.estado, t1_0.fecha, t1_0.tipo
from
    tramite t1_0
<<<<<< FIN DEL CONTEO – 6 trámites traídos
(no se tocó el contribuyente de ninguno)

```

Un solo Hibernate: Un viaje para seis trámites.

Ahora **cámbialo a EAGER** en Tramite.java y vuelve a arrancar:

```
@ManyToOne(fetch = FetchType.EAGER)
```

```

>>>>> EMPIEZA EL CONTEO
Hibernate: ...
Hibernate: ...
Hibernate: ...
Hibernate: ...
<<<<<< FIN DEL CONTEO – 6 trámites traídos

```

Cuatro. Uno por los trámites y tres más para traer contribuyentes **que nadie miró**.

Vuelve a dejarlo en LAZY antes de seguir.

Salen cuatro y no siete porque los seis trámites son de sólo tres contribuyentes distintos, y el archivista no baja dos veces a por la misma ficha. Si cada trámite fuera de uno distinto, serían siete. Ése es el problema que el Lab 06 mide en grande.

Vas bien si...

Contaste **1** con LAZY y **4** con EAGER, y volviste a dejar LAZY.

Si te atascas

1 · Cuentas lo mismo en los dos casos.

No reiniciaste después de cambiar la anotación, o editaste el archivo de `solucion/` en vez del de `practica/`.

2 · cannot find symbol: FetchType

Falta `import jakarta.persistence.FetchType;`

3.4 Paso 4 · El error que todo el mundo se encuentra

Qué vamos a hacer

Provocar a propósito el `LazyInitializationException`. **Este paso está pensado para fallar.**

Para entenderlo mejor

Le pediste al archivista el trámite, **te fuiste de la oficina**, y desde la calle le gritas que te suba también al contribuyente. Ya no puede: **cerró**.

LAZY significa «lo traigo cuando lo pidas» — y eso sólo vale **mientras la sesión con la base sigue abierta**. Fuera de la transacción, el objeto que tienes en la mano es un cascarón que ya

no sabe volver a la base.

El problema

Es el precio de LAZY, y hay que conocerlo antes de encontrárselo: el código compila, arranca, funciona en las pruebas del método donde hay transacción, y **revienta cuando alguien lo llama desde fuera**.

La alternativa, y por qué no

- **Poner EAGER** para que no pase: quita el error y trae de vuelta el problema del paso 3. Es cambiar un fallo ruidoso por uno silencioso y caro.
- **open-in-view: true**, que deja la sesión abierta hasta que termina la petición HTTP: hace desaparecer el error, y a cambio esconde consultas dentro del renderizado de la respuesta. Está **apagado a propósito** en este curso.
- **Traer lo que hace falta dentro de la transacción**, que es la respuesta correcta y la del Lab 06: se pide de una vez con un JOIN FETCH.

Lo que vas a ver

```
=== 5 · LazyInitializationException · fuera de la transacción ===
  trámite cargado (y la sesión ya se cerró): Tramite{id=1, tipo='Declaración F29',
  ↪ ...}
  REVENTÓ, y está bien: LazyInitializationException
  mensaje: Could not initialize proxy [cl.dgt.relaciones.entities.Contribuyente#1] -
  ↪ no session
```

Lee el mensaje entero: «*no session*». No dice que el contribuyente no exista — dice que **ya no hay conexión** para ir a buscarlo.

Vas bien si...

Sale `LazyInitializationException` con el texto `- no session`, y la aplicación **sigue** con la demo siguiente. Este error está capturado a propósito.

Si te atascas

1 · No revienta: trae el contribuyente sin quejarse.

Tienes EAGER puesto del paso 3, o `open-in-view` encendido. Con EAGER no puede fallar, porque el contribuyente ya venía.

2 · Te sale este error en TU código, fuera de este paso.

Es lo normal la primera vez. Tres salidas, de mejor a peor: traerlo dentro de la transacción con JOIN FETCH (Lab 06); mover la lógica dentro de un método `@Transactional`; o —última— cambiar a EAGER, sabiendo lo que cuesta.

3.5 Paso 5 · Un nombre de método que cruza la relación

Qué vamos a hacer

Pedir los trámites de un RUT — que es un dato **del contribuyente**, no del trámite.

Para entenderlo mejor

«*Tráeme los trámites del contribuyente con este RUT*». El archivista tiene que mirar en **los dos cajones**: buscar la ficha del contribuyente por su RUT, y traer los trámites que la apuntan. Tú sólo se lo pides.

El problema

El RUT no está en la tabla tramite. Un `findByRut` no puede funcionar, porque el trámite no tiene esa casilla.

La alternativa, y por qué no

- **Dos consultas**: buscar el contribuyente por RUT y después sus trámites. Funciona y son dos viajes donde basta uno.
- **@Query con JPQL** escrito a mano: control total, y hay que escribirlo.
- **Una consulta derivada que navega**, que es lo de aquí: `findByContribuyenteRut` — «por el contribuyente, su rut». Spring Data entiende el camino y arma el join.

Se pega

En `practica/src/main/java/cl/dgt/relaciones/repositories/TramiteRepository.java`:

```
package cl.dgt.relaciones.repositories;

import cl.dgt.relaciones.entities.Tramite;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface TramiteRepository extends JpaRepository<Tramite, Long> {

    List<Tramite> findByContribuyenteRut(String rut);
}
```

Lo que vas a ver

```
=== 6 · CONSULTA DERIVADA QUE NAVEGA · findByContribuyenteRut() ===
Hibernate:
    select
        t1_0.id, t1_0.contribuyente_id, t1_0.estado, t1_0.fecha, t1_0.tipo
    from
        tramite t1_0
    join
        contribuyente c1_0
        on c1_0.id=t1_0.contribuyente_id
    where
        c1_0.rut=?
trámites del RUT 76.543.210-K -> 2
```

Un solo SELECT, con el join dentro. Nadie escribió ese join: salió del nombre del método.

Vas bien si...

La consulta que sale tiene un join y un where `c1_0.rut=?`, y devuelve 2 trámites.

Si te atascas

1 · No property 'contribuyenteRut' found

Spring Data no encontró el camino. Si hubiera ambigüedad, se desambigua con un guion bajo: `findByContribuyente_Rut`.

2 · Devuelve 0 trámites.

El RUT que pasas no coincide exactamente con el guardado — los puntos y el guion cuentan.

4 Lo que aprendiste

1 · Una relación se declara en el lado que tiene la columna.

`@ManyToOne` y `@JoinColumn` van donde está la casilla. El otro lado, si lo hay, usa `mappedBy` para decir que la verdad vive enfrente — y `mappedBy` nombra el **campo**, no la columna.

2 · LAZY y EAGER no cambian lo que el código hace: cambian cuántas veces va a la base.

Lo contaste: **1 consulta con LAZY, 4 con EAGER**, para los mismos seis trámites. Y la especificación pone EAGER por defecto en `@ManyToOne`, así que si no lo escribes, tienes el caro.

3 · LAZY tiene un precio, y se llama `LazyInitializationException`.

Fuera de la transacción, lo que no se trajo ya no se puede traer. La respuesta buena no es poner EAGER: es pedir lo que hace falta mientras la sesión está abierta.

4 · El nombre de un método puede cruzar una relación.

`findByContribuyenteRut` genera un join que nadie escribió. Es cómodo hasta que el nombre deja de caber en una línea — ahí toca `@Query`.

5 Para profundizar

- **Añade** `findByContribuyenteRutAndEstado` y mira el join con dos condiciones.
- **Pon** `@OneToMany(fetch = FetchType.EAGER)` en el contribuyente y cuenta otra vez. ¿Cuántas consultas salen ahora al listar contribuyentes?
- **Prueba** `orphanRemoval = true` en el lado inverso y borra un trámite de la lista. Mira qué hace. Después quítalo: en material tributario, un borrado en cascada silencioso es peligroso.
- **Cuenta las consultas al listar los 3 contribuyentes** y tocar sus trámites. Ese número es el problema del Lab 06.

6 Antes de cerrar

Párala con `Ctrl+C`.

```
./mvnw clean
```

Lo que te llevas:

La columna vive en un lado y mappedBy en el otro. LAZY trae lo que pidas cuando lo pidas — mientras la sesión siga abierta—; EAGER trae siempre, y se paga en consultas.

Lo que queda pendiente, y abre el Lab 06: hoy contaste 1 contra 4 con seis trámites. Con doscientos contribuyentes ese «4» se convierte en **201 consultas para una sola pantalla**. En el Lab 06 se mide, se ve y se arregla de tres formas distintas.